26.2.26 Experiment : 01 Monoalphabetic cipher.

Aim

To implement a Monoalphabetic cipher in python for performing encryption and decryption of text using a predefined cipher mapping.

Algorithm :

* Initialize plain and cipher alphabetic
* Read user choice
* Read input text
* Initialize empty string result.
* If choice is E, replace each letter using plain -> cipher.
* If choice is D, replace each letter using cipher -> plain
* Keep non-alphabetic characters unchange
* Display the output text.

program :

```
plain = "abcdefghijklmnopqrstuvwxyz"
cipher = "qwertyuiopasdfgjklzxcvbnm"
choice = input ("Enter E for Encryption or D for Decryption) upper()
text = input ("Enter text :"). lower()
result =
if choice == "E":
    for ch in text:
```

```
result + = cipher [plain. index (ch)]
else :
    result + = ch
print (" Encrypted text: ", result)
else :
    print (" Invalid choice")
```

output:

```
Enter E for Encryption or D for Decryption.
Enter text : hello world
Encryption text : it ssg vgk sr

Enter E for Encryption or D for Decryption : D
Enter text : it ssg vgk sr
Decrypted text : hello world
```

Result:

Thus the program executed successfully

28.2.26

# Experiment -6
## Data Encryption standard

AIM:
To write a python program on data encryption standard algorithm.

Algorithm:
Import des from crypto library
Declare variables [choice, key]
Initialize cipher as des.new, key.encode(),
des.mode.
Declare condition of if & else
if choice is, then
Give the encryption message
encrypted-text = cipher.encrypt(text)
elif choice is 2 then
Given value to decrypt
decrypted_text = unpad (cipher
decrypt (encrypted-bytes),
des.block-size)

else then
print invalid choice
Gain the input & run to get the output.

code:
```
from crypto.cipher import DES
from crypto.util.padding import pad,unpad
print("1.Encrypt")
print("2. Decrypt")
choice = input("Enter your choice (1/2): ")
key = input("Enter 8-character key: ")
```

6/3/26 Experiment - 11B tream cipher.

AIM:
To perform encryption and decreption using stream cipher.

Algorithm:
* Declare variable text, key, result, i
* Get key and plain text "Krithiya"
* print result = chr (ord (text [i]) ^ ord [K]) performing
* call the stream cipher function with text key
* print the output.

Code:

```
def stream_cipher (text, key):
    result = " "
    For i in range (len (text)):
        k = key [i % len (key)]
        result + = chr (ord (text [i] ^ ord (k))
    return result
choice = input ("Enter E for Encryption or D for Decryption : ").upper().
Text = input ("Enter text : ")
key = input ("Enter key : ")
output = stream_cipher (text, key)
print ("Result", output)
```

Output:

```
Enter E for Encryption or D for Decryption:
Enter text : Krithiya
Enter key : 3
Result :
```

Experiment - 10 RC5

10/3/26

AIM:

To write a python program to perform encryption and decryption using RC5 cryptoigraphic technique.

Algorithm:

* Define function rotl() for left rotation and rotr() for right rotation
* Read input as "krittiya"
* Initialize empty string result.
* For each character in text. convert char into ASCII value
  x = rotl(x ^ key, key % w, w)
* if choice is decryption : x = rotr(x, key % w, w)
* convert ASCII value back to character and store in result.
* Display encrypted and decrypted text.

Code:

```
def rotl(x, y, w):
    return ((x << y) & (2 ** w - 1)) |
(w >> (w - y))
def rotr(x, y, w):
    return (x >> y) | ((x << (w - y)) &
(2 ** w - 1))
choice = int(input("Enter choice (1 encrypt
2 decrypt): "))
text = input("Enter text: ")
key = int(input("Enter rounds: "))
```

Experiment -9 Triple DES

4.03.26

AIM:
To encrypt or decrypt a message using triple DES

Algorithm:
* Declare the variables to stores the text, k1, k2, k3, choice & loop counter.
* Display the Menu for Encrypt and decrypt
* Read the users choice
* Read three keys.
* If choice is 1, perform XOR operation on each character
Text [i] = text [i] ^ k1 ^ k2 ^ k3
* If choice is 2, perform XOR again
* Display the result.

code:

```
# include <stdio.h>
# include <string.h>
int main () {
  char text [100];
  char k1, k2, k3;
  int choice, i;
  print ("1. Encrypt \n 2. Decrypt \n Enter you
          choice :")
  Scanf ("%d", & choice);
  get char ();
  printf ("Enter text :");
  F gets (text, 100, std in);
  printf ("Enter key 1 :");
  Scanf ("%c", & key 1);
```

03.03.26

# Experiment - 8

## RSA Algorithm.

AIM:
To implement RSA algorithm to encryt & decrypt a plain text

Algorithm:
choose two prime numbers p and q
compute $n = p \times q$, $\varphi(n) = (p-1)(q-1)$
choose public key such that
$1 < e < \varphi(n)$ & $\gcd(e, \varphi(n)) = 1$
compute private key d such that $(e \times d) \bmod (n) = 1$
Encryption : cipher text $c = m^e \bmod n$
Decryption : original message $m = c^d \bmod n$

code :

```
p = 3
q = 11
n = p * q
phi = (p-1) * (q-1)
for i in range (1, phi)
    if (e*i) % phi == 1:
        d = i
        break
print ("public key (e, n) : ", (e, n))
print ("private key (d, n) : ", (d, n))
if choice == 'E':
    msg = int (input ("Enter message to encryption
elif choice == 'D'
    cipher = int (input ("Enter number to decrypt : "
    message = pow (cipher, d, n)
    print ("message : ", message)
```

```
result = ''
rail_index = [0] * rails
for r in pattern
    result += fence[r][rail_index[r]]
    rail_index[r] += 1
return result
choice = input("Type E to encrypt or D decrypt:").upper()
if choice == 'E':
    text = input("Enter plain text: ")
    print("cipher text")
```

Result:

Thus the program was executed successfully.

27.2.26 Experiment -3 Vigenere cipher

Aim:
To implement the vigenere cipher for encryption and decryption of a given plain text.

Algorithm:
* Read the plain text / cipher text and keyword from the user
* Repeat the keyword to match the length of the text
* convert both text and keyword to uppercase
* For each character:
  Encryption: add the positions of plain text and key letters.
  Decryption: subtract the key letterposition from the ciphertext letter.

Code:

```
def vigenere (text, key, mode):
    result = " "
    key = key.upper()
    for i in range (len (text)):
        if text [i] . is alpha ():
            if mode == " e ":
                val = (ord (text [i].upper()) + (key [i % len(key)] - 2*65) % 26
            else:
                val = (ord (text[i] .upper() - ord (key [i % len(key)]
            result += chr (val + 65)
    output = vigenere (text , key , choice)
    print ("Result", output).
```

```
if choice == "E"
    print ("Encrypted text = ", result)
elif choice == "D"
    print ("Decrypted text = ", result)
else
    print ("Invalid choice")
```

Output:

```
Enter text : hello
Enter shift value : 3
Enter E for Encryption or D for Decryption : E
Encrypted text : khoor

Enter text : khoor
Enter shift value : 3
Enter E for Encryption or D for Decryption : D
Decrypted text : hello
```

Result:

Thus the program executed successfully

26.2.26 Experiment -2 Caesar cipher.

Aim:

To implement caesar cipher for encryption and decryption of text using a shift value.

Algorithm:

* Read input text from the user
* Read shift value
* Read choice (E for Encryption / D for Decryption)
* Initialize empty string result.
* For each character in the text
  * check if it is an alphabet
  * For Encryption, shift character forward
  * keep non-alphabet character unchanged
* Display encrypted or decrypted text
* If invalid choice entered display error manage

code:

```
text = input ("Enter text : ")
shift = int (input ("Enter shift value : "))
choice . input ("Enter E for Encryption or D for
Decryption:") . upper()
result : "
for ch in text :
    if ch. is alpha ()
    if choice = = "E"
        result += chr ((ord (ch). a+ shift ) %. 2
    elif choice = = "D":
result += chr ((ord (ch) - a+ shift ) %. 26 + 77)
```

27.2.26

# Experiment - 4 Transposition cipher Rail Fence Transposition cipher.

**Aim:**

To implement a Transposition cipher that encrypts and decrypts a given text.

**Algorithm:**

* start the program
* Get the input text from the user
* Get the numeric key value
* Ask the user to choose encryption (E) or decryption (D)
* If encryption is selected
* arrange the text based on the key
* Read characters at fixed intervals to generate cipher text
* Rearrange and read row wise to obtain original text
* Display the result
* stop the program

**code:**

```
def rail-fence-encrypt(text, rails=2):
    fence = [''for- in range(rails)]
    rail, direction =0,1
    For ch in text:
        fence[rail] += ch
        rail += direction
        if rail == 0 or rail == rails -1:
            direction = -1
fence = [''for - in range(rails)]
idx = 0
for r in range(rails):
    for i in range(len(cipher)):
        if pattern[i] == r:
```

28.2.26

# Experiment 5
## Hill cipher

AIM:
To implement the python program for implementing hill cipher

Algorithm:
* Declare variable & import numpy library
* Initialize conditional statement containing if and else
* if choice is decryption:
  plain = np.dot (inv-key, pair) % 26
* Give input for text, & choice
* print the output as result

code:

```
import numpy as np
text = input ("Enter text:").replace(" ","").upper()
choice = input ("Enter E for Encryption or D for Decryption:").lower()
key = np.array([[3,3],[2,5]])
if len(text)%2!=0:
    text += 'x'
result = ""
if choice == 'e':
    for i in range(0, len(text), 2):
        pair = np.array([[ord(text[i])-65],[ord(text[i+1])-65]])
        cipher = np.dot(key, pair)%26
        result += chr(cipher[0][0]+65)+chr(cipher[1][0]+65)
```

output:

Enter text : HI

Enter, E for Encryption or D for decryption : E

Result : TC

Enter text : TC

Enter E for encryption or D for Decryption

Result : HI

Result:

Thus the program implemented successfully.

```
if len (key) != 8 :
    print ("Key must be exactly 8 characters")
    exit()

cipher = DES.new (key.encode(), DES.MODE_ECB)
if choice == "1" :
    message = input ("Enter message to encrypt:")
    padded_text = pad (message.encode(), DES.block_size)

elif choice == "2" :
    encrypted_hex = input ("Enter hex value to decrypt:")
    encrypted_bytes = bytes.fromhex (encrypted_hex)
    decrypted_text = unpad (cipher.decrypt (encrypted_bytes), DES.block_size)
    print (decrypted_text)
else :
    print ("Invalid choice")
```

Output:

1. Encrypt
2. Decrypt

Enter your choice (1/2) : 1
Enter 8-character key : 12345678
Enter message to encrypt : HELLO
Encrypted (hex) : 8bba144112de58903

Result:

Thus the program executed successfully.

else :
print ( "Invalid choice " )

output :

Enter E or D : E
Enter number to encrypt : 5
Encrypted message : 26
Enter E or D : D
Enter number to decrypt : 26
Decrypt message : 5

Result :
Thus the program was implemented Successfully.

```
printf("Enteing key 2 : ");
scanf("%c", &key 2 : ");
printf("Enter key 3 : ");
scanf("%c", &key 3);
if (choice ==1) {
for (i=0; i< strlen (text); i++)
    text [i] = text [i] ^ K1 ^ K2 ^ K3;
    printf("Encrypted Text : %s", text);
}
elseif (choice ==2) {
    for (i=0, i<strlen (text); i++)
        text[i] = text [i] ^ K3 ^ K2 ^ K1;
        print ("decrypted text : %s", text);
}
else
    printf("Invalid choice");
return 0;
}
```

~~Result~~:

output :

```
1. encrypt
2. Decrypt
Enter choice : 1
Enter text : krithiya
Enter key : 3
Enter key : 4
Enter key : 3 : 5
Encrypted Text : Y@[.F2[KS8
```

Result :

Thus the program for encryption & decryption using Triple DES has been executed successfully.

```
rounds = int (input ("Enter rounds : "))
w = int (input ("Enter word size : "))
result = " "
for ch in text :
    x = ord (ch)
    if choice == 1:
        for i in range (rounds):
            x = rotl (x ^ key, key % w, w)
    elif choice == 2:
        for i in range (rounds):
            x = rotr (x, key % w, w) ^ key
    result += chr(x)
if choice == 1:
    print ("Encrypted text:", result.encode().hex())
else :
    print ("Decrypted Text :", result)
```

output :

```
Enter choice (1 Encrypt / 2 decrypt) : 1
Enter text : Krithiya
Enter key : 4
Enter rounds : 3
Enter word size : 8
Encrypted text : c2b223c2924 3c282c
                 292c29312
Enter choice (1 Encrypt / 2 Decrypt) : 2
Enter encrypted text : c2b223c2924 3c282c
                       292c29312
Enter key : 4
Enter rounds : 3
Enter word size : 8
Decrypted text : Krithiya
```

Result :

Thus the program implemented successfully.

7/3/26

# Experiment - 12 Blow Fish Algorithm

Aim:
To perform encryption and decryption using Blow fish algorithm.

Algorithm:
* Read inputs plaintext and secret key
* Initialize P-array and S-boxes and modify them using the key
* Divide the 64 bits plaintext block into two values L and R
* Apply XOR function
  * L = L XOR P[i]
  * R = F[L] XOR R
  * Swap (L, R)
* Combine both values and print the cipher text.

code:
```
from crypto.cipher import Blowfish
from crypto.util.padding import pad, unpad
key = input("Enter E for Encryption or
D for Decryption:").upper()
if choice == "E":
    text = input("Enter text to encrypt:")
    cipher = Blowfish.new(key, Blowfish.MODE_ECB)
    encrypted = cipher.encrypt(pad(text.encode(),
Blowfish.block_size))
    print("Encrypted text:", encrypted.hex())
```

Enter E for Encryption or D for Decryption: d

Enter text :

Enter key : 3

Result : Kmithiya

Result:

Thus the program for stream cipher is implemented successfully.

Experiment - 7 Diffie Hellman Algorithm

03.03.26

AIM :
To encrypt or decrypt a message using Diffie - Hellman Algorithm.

Algorithm:
* Declare the variables p, g, a, b, x, y, key, key
* Initialize values p = 23, g = 5, a = 6, b = 5
* public keys are computed x = $g^a$ mod p, y = $g^b$ mod p
* shared secret key is generated by key = $y^a$ mod p
* Taking input as "KRITHIYA" encrypted = ASCII + key
* Decrypted = ASCII - key

code :
```
p = int (input ("Enter prime number (p): "))
g = int (input ("Enter primitive root (g): "))
a = int (input ("Enter private key of Sender: "))
b = int (input ("Enter private key of receiver: "))

x = pow (g, a, p)
y = pow (g, a, p)

key = pow (y, a, p)
print ("shared secret key : ", key)
choice = input ("Enter E to encrypt and D to Decrypt: ").upper()
if choice == 'E':
    msg = input ("Enter msg : ")
```

```
elif choice == "D":
    text = bytes.fromhex(input("Enter encrypted
    text (hex): "))
    cipher = Blowfish.new(key, Blowfish.MODE_ECB)
    decrypted = unpad(cipher.decrypt(text),
    Blowfish.block_size)
    print("Decrypted text:", decrypted.decode())

else:
    print("Invalid choice")
```

Output:

```
Enter Key (4-56 characters):
Enter E for Encryption or D for decryption: E
Enter text to encrypt: krithiya
Encrypted text: 1d805c1808ffa4b47328 5ab
                                     795ed
```

EXP.NO:14 MAC (Message Authentication code) 13/3/26

Aim: To implement message Authentication code to verify data integrity and authenticity of a message using a secret key.

Algorithm:

1) Import required libraries (hmac, hashlib)
2) Take the message and secret key as input.
3) convert the message and key into byte format.
4) Generate the MAC value using the hashing algorithm
5) convert the MAC value into hexadecimal format.

Program:

```
import hmac
import hashlib
message = input("Enter the message:")
key = input("Enter the secret key:")
mac = hmac.new(key.encode(), message.encode(), hashlib.sha256)
print("Generated MAC:", mac.hexdigest())
```

output:

Enter the message: hello.
Enter the secret key: key123
Generated MAC: 3d41402abc4b2a76b 9719d911017c592

Result: The MAC code is implemented successfully

Experiment - 13 SHA

AIM: To implement the Secure Hash - Algorithm (SHA-256) to generate a fixed length hash value for a given input message

Algorithm:

* Import the hastilib library
* Read the input message from the user
* convert the input message into byte format using encode()
* Apply the SHA-256 hashing function using hashlib SHA-256()
* Display the generated SHA-256 hash value

program:

```
import hashlib
msg = input ("Enter the message:")
hash_value = hashlib.sha256(msg.encode())
print ("SHA-256 Hash:", hash_value.hexdigest
()
```

Output:

Enter the message : hello
SHA 256
Hash : 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824.

Result: The SHA is successfully implemented

```
print("Encrypted:", enc)
elif choice == 'D':
    msg = input("Enter Encrypted msg")
    decrypted = ''.join(chr(ord(c) - key))
    print("Decrypted:", decrypted)

else:
    print("Invalid choice")
```

output:

Enter prime number (p): 3
Enter primitive root (g) = 7
Enter private key of A : 2
Enter private key of B : 4
Shared secret key : 1
Enter E for Encryption or D for Decryption:
Encrypted : q sb s ubob

Enter prime number (p): 3
Enter primitive root (g): 7
Enter private key of A : 2
Enter private key of B : 4
Shared secret key : 1
Enter E for Encryption or D for Decryption : D
Enter message : q sbsubob
Decrypted : KRITHIYA

Result:

Thus the program implemented Successfully.

output:

Encrypt or Decrypt(e/d): e
Enter text: hello world
Enter key: key
Result: RIJVSGSPVH

Result:
Thus the program is executed successfully